Chương 4 – Thuật toán tham lam (Greedy Algorithms): Mục 4.1–4.9

Oct 24, 2026 · @Hung Ta Quang

Thuật toán tham lam xây dựng lời giải từng bước, mỗi bước chọn phương án **trông tốt nhất ngay lúc đó** và không bao giờ quay lại sửa. Chương 4 của *Algorithm Design* (Kleinberg & Tardos) chỉ ra khi nào cách làm "thiển cận" này vẫn cho lời giải tối ưu, và dạy hai kỹ thuật chứng minh điều đó.

Ghi chú: trong giáo trình này, Chương 4 là *Greedy Algorithms*; *Dynamic Programming* là Chương 6. Tài liệu trình bày lại từ kiến thức về giáo trình, ví dụ số và hình vẽ là minh hoạ tự dựng; số thứ tự mệnh đề nên đối chiếu với bản sách. Mọi công thức được viết bằng ký tự thường (Σ, ≤, ·, chỉ số dạng v_i) để không bị lỗi khi tải về Word hoặc PDF.

Ý tưởng cốt lõi qua một hình ảnh

Hãy hình dung bạn trả lại tiền thừa 63.000 đồng bằng ít tờ nhất, với các mệnh giá 50.000, 20.000, 10.000, 5.000, 2.000, 1.000. Cách tự nhiên là luôn lấy tờ **lớn nhất không vượt quá số còn lại**: 50.000 + 10.000 + 2.000 + 1.000 = 4 tờ. Với hệ mệnh giá này, cách tham lam đó là tối ưu.

Nhưng nếu hệ mệnh giá là {1, 3, 4} và cần trả 6, tham lam lấy 4 + 1 + 1 = 3 đồng xu, trong khi tối ưu là 3 + 3 = 2 đồng xu. Bài học: **tham lam đúng hay sai phụ thuộc vào cấu trúc bài toán**, và ta phải chứng minh chứ không được tin trực giác.

Hai kỹ thuật chứng minh xuyên suốt chương

Kỹ thuật	Ý tưởng	Hình ảnh	Dùng ở
Tham lam luôn đi trước (*greedy stays ahead*)	So sánh lời giải tham lam với một lời giải tối ưu bất kỳ, từng bước một; chứng minh sau mỗi bước, tham lam không bao giờ thua	Hai vận động viên chạy cùng đường; ở mọi cột mốc, người tham lam luôn ở phía trước hoặc ngang bằng	4.1, 4.4
Lập luận trao đổi (*exchange argument*)	Lấy một lời giải tối ưu bất kỳ, biến đổi dần từng chút thành lời giải tham lam mà không làm chất lượng giảm	Đổi chỗ từng cặp sách trên giá cho đến khi giá sách giống hệt giá của người tham lam, mà không làm giá trị giảm	4.2, 4.3, 4.5, 4.8

Ngoài ra, Chương 4 còn giới thiệu các cấu trúc dữ liệu giúp cài đặt tham lam nhanh: hàng đợi ưu tiên (4.4, 4.5) và Union-Find (4.6).

Lộ trình chín mục

Mục	Bài toán	Quy tắc tham lam	Thời gian
4.1	Lập lịch khoảng	Chọn yêu cầu kết thúc sớm nhất	O(n log n)
4.2	Tối thiểu độ trễ	Làm việc có hạn chót sớm nhất trước	O(n log n)
4.3	Bộ nhớ đệm tối ưu	Loại phần tử được dùng lại xa nhất trong tương lai	—
4.4	Đường đi ngắn nhất	Mở rộng đỉnh gần nguồn nhất (Dijkstra)	O(m log n)
4.5	Cây khung nhỏ nhất	Thêm cạnh rẻ nhất an toàn (Kruskal, Prim)	O(m log n)
4.6	Union-Find	Hợp tập nhỏ vào tập lớn	O(log n) mỗi Find
4.7	Phân cụm	Kruskal dừng khi còn k cụm	O(m log n)
4.8	Mã Huffman	Gộp hai ký tự tần suất thấp nhất	O(k log k)
4.9	Cây khung có hướng	Chọn cạnh vào rẻ nhất, co chu trình	O(mn)

4.1 Lập lịch khoảng: tham lam luôn đi trước

Chọn yêu cầu **kết thúc sớm nhất** rồi loại mọi yêu cầu xung đột với nó, lặp lại, cho số yêu cầu được phục vụ lớn nhất; chứng minh bằng kỹ thuật "tham lam luôn đi trước".

Phát biểu bài toán

Có n yêu cầu dùng một tài nguyên; yêu cầu i chiếm khoảng [s(i), f(i)). Hai yêu cầu **tương thích** nếu không chồng lấn. Tìm tập tương thích có **nhiều yêu cầu nhất** (khác 6.1: ở đây mọi yêu cầu có giá trị bằng nhau).

Hình ảnh: một phòng họp duy nhất, nhiều cuộc họp xin giờ; muốn tổ chức được càng nhiều cuộc họp càng tốt.

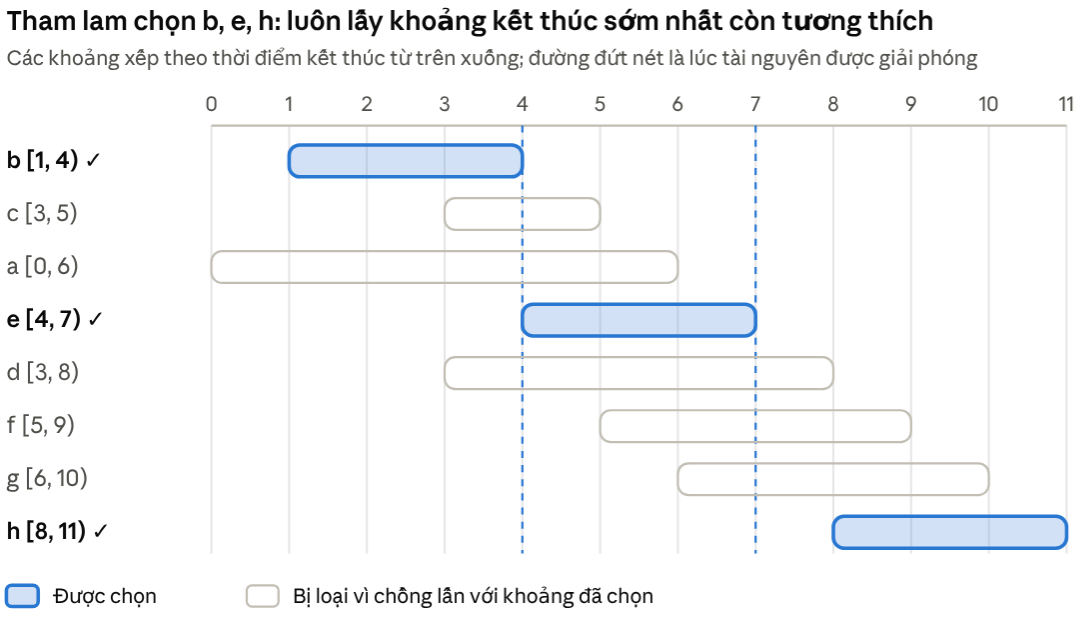
Bốn quy tắc tham lam "hợp lý" — và ba quy tắc sai

Quy tắc	Ý tưởng	Phản ví dụ
Bắt đầu sớm nhất	Dùng tài nguyên càng sớm càng tốt	Một yêu cầu bắt đầu sớm nhưng kéo dài cả ngày, chặn mọi yêu cầu khác
Khoảng ngắn nhất	Chiếm ít thời gian nhất	Một khoảng ngắn nằm đè lên chỗ nối của hai khoảng dài tương thích nhau: chọn nó thì mất cả hai
Ít xung đột nhất	Chặn ít yêu cầu khác nhất	Có cấu hình (trong sách) khiến yêu cầu ít xung đột nhất không thuộc lời giải tối ưu
Kết thúc sớm nhất	Giải phóng tài nguyên sớm nhất	Không có — đây là quy tắc đúng

Thuật toán

```
R ← tập mọi yêu cầu;  A ← rỗng
while R chưa rỗng
   chọn i ∈ R có f(i) nhỏ nhất
   thêm i vào A
   xoá khỏi R mọi yêu cầu không tương thích với i
return A
```

Ví dụ. Tám yêu cầu: a [0, 6), b [1, 4), c [3, 5), d [3, 8), e [4, 7), f [5, 9), g [6, 10), h [8, 11). Xếp theo thời điểm kết thúc: b (4), c (5), a (6), e (7), d (8), f (9), g (10), h (11). Tham lam chọn b; bỏ c, a, d vì chồng lấn; chọn e (bắt đầu lúc 4 ⩾ 4); bỏ f, g; chọn h (bắt đầu lúc 8 ⩾ 7). Kết quả A = {b, e, h}, 3 yêu cầu.



Hình 4.1 · tám khoảng và lựa chọn tham lam

Đọc hình từ trên xuống: mỗi khi một khoảng được chọn, mọi khoảng bắt đầu trước đường đứt nét của nó đều bị loại; khoảng được chọn tiếp theo là khoảng đầu tiên còn lại nằm bên phải đường đó.

Chứng minh: tham lam luôn đi trước

Gọi A = {i_1, …, i_k} là lời giải tham lam và O = {j_1, …, j_m} là một lời giải tối ưu, cả hai xếp theo thứ tự thời gian. Ta muốn chứng minh k = m.

Bổ đề (đi trước). Với mọi r ⩽ k:

```
f(i_r) ⩽ f(j_r)
```

Chứng minh quy nạp theo r. Với r = 1: tham lam chọn yêu cầu kết thúc sớm nhất nên f(i_1) ⩽ f(j_1). Giả sử f(i_(r−1)) ⩽ f(j_(r−1)). Vì O tương thích, j_r bắt đầu sau khi j_(r−1) kết thúc: s(j_r) ⩾ f(j_(r−1)) ⩾ f(i_(r−1)). Vậy j_r vẫn còn trong R khi tham lam chọn phần tử thứ r; tham lam chọn cái kết thúc sớm nhất nên f(i_r) ⩽ f(j_r).

Định lý. A là tối ưu. *Chứng minh phản chứng:* nếu m > k thì O có thêm j_(k+1), bắt đầu sau f(j_k) ⩾ f(i_k). Như vậy j_(k+1) tương thích với mọi phần tử của A, nên R chưa rỗng khi tham lam dừng — mâu thuẫn.

Hình ảnh: hai người cùng xếp lịch; ở mỗi lượt chọn, "đồng hồ" của người tham lam luôn chỉ thời điểm sớm hơn hoặc bằng, nên người ấy không bao giờ hết chỗ trước.

Cài đặt O(n log n)

```
def interval_scheduling(intervals):
    """intervals: danh sách (s, f). Trả về các khoảng được chọn."""
    chosen, last_finish = [], float('-inf')
    for s, f in sorted(intervals, key=lambda x: x[1]):   # O(n log n)
        if s >= last_finish:                               # tương thích với
khoảng vừa chọn
            chosen.append((s, f)); last_finish = f
    return chosen

print(interval_scheduling([(0,6),(1,4),(3,5),(3,8),(4,7),(5,9),(6,10),
(8,11)]))
# [(1, 4), (4, 7), (8, 11)]   -> b, e, h
```

Bài toán liên quan: phân hoạch khoảng (Interval Partitioning)

Bây giờ phải phục vụ **tất cả** yêu cầu, dùng nhiều tài nguyên (nhiều phòng học), sao cho số phòng **ít nhất**.

Độ sâu d của tập khoảng là số khoảng lớn nhất cùng đi qua một thời điểm. Hiển nhiên cần ít nhất d phòng, vì d lớp đó phải ở d phòng khác nhau. Điều bất ngờ là **d phòng luôn đủ**:

```
Xếp các khoảng theo thời điểm bắt đầu
for mỗi khoảng I_j theo thứ tự đó
   loại các nhãn (phòng) đang bị các khoảng trước đó chồng lấn với I_j chiếm
   nếu còn nhãn trong {1, ..., d}: gán một nhãn còn trống cho I_j
```

Khi xét I_j, các khoảng trước chồng lấn với nó đều đi qua thời điểm s(I_j), nên có tối đa d − 1 khoảng như vậy; luôn còn ít nhất một phòng trống. Vậy thuật toán dùng đúng d phòng — đạt cận dưới, nên tối ưu. Đây là kiểu lập luận **cận dưới cấu trúc**: tìm một "vật cản" mà mọi lời giải phải vượt, rồi chỉ ra tham lam vượt đúng bằng nó.

4.2 Lập lịch tối thiểu độ trễ: lập luận trao đổi

Xếp các công việc theo **hạn chót tăng dần** (Earliest Deadline First, EDF), làm liền nhau không nghỉ, cho độ trễ lớn nhất nhỏ nhất; chứng minh bằng cách đổi chỗ dần một lịch tối ưu cho đến khi nó trùng với lịch tham lam.

Phát biểu bài toán

Một máy, n công việc. Công việc i cần t_i đơn vị thời gian và có hạn chót d_i. Một lịch gán thời điểm bắt đầu s(i), kết thúc f(i) = s(i) + t_i, các việc không chồng lấn. Độ trễ của i:

```
l_i = max( 0 ,  f(i) − d_i )
```

Mục tiêu: tối thiểu **độ trễ lớn nhất** L = max_i l_i. Hình ảnh: một kỹ thuật viên phải sửa nhiều sự cố, mỗi sự cố có hạn xử lý; anh muốn sự cố bị trễ nhiều nhất bị trễ ít nhất có thể.

Hai quy tắc sai

Quy tắc	Phản ví dụ (t, d)	Kết quả
Việc ngắn trước	Việc 1: (1, 100); việc 2: (10, 10)	Làm 1 trước thì việc 2 xong lúc 11, trễ 1; làm 2 trước thì không trễ
Độ chùng d − t nhỏ trước	Việc 1: (1, 2); việc 2: (10, 10)	Độ chùng 1 và 0, làm 2 trước thì việc 1 xong lúc 11, trễ 9; làm 1 trước chỉ trễ 1

Quy tắc đúng lại **bỏ qua hoàn toàn độ dài t_i**: chỉ nhìn hạn chót.

Thuật toán EDF và ví dụ

```
Xếp các việc theo hạn chót: d_1 ≤ d_2 ≤ ... ≤ d_n
f ← 0
for i = 1 to n
   s(i) ← f;   f(i) ← s(i) + t_i;   f ← f(i)
```

Việc	t	d	Bắt đầu	Kết thúc	Độ trễ
A	3	6	0	3	0
B	2	8	3	5	0
C	1	9	5	6	0
D	4	9	6	10	**1**
E	3	14	10	13	0
F	2	15	13	15	0

Độ trễ lớn nhất L = 1. Bốn việc A, B, C, D có hạn chót ≤ 9 nhưng tổng thời lượng là 3 + 2 + 1 + 4 = 10 > 9 — không lịch nào tránh được trễ, nên 1 là tối ưu.

Chứng minh bằng lập luận trao đổi

Gọi **nghịch thế** là cặp việc i, j mà i được làm trước j nhưng d_j < d_i. Lịch EDF không có nghịch thế và không có thời gian rỗi. Chứng minh qua bốn bước:

1. **Có lịch tối ưu không có thời gian rỗi.** Nếu máy nghỉ, kéo các việc sau lên sớm hơn; độ trễ không tăng.

2. **Mọi lịch không nghịch thế, không rỗi đều có cùng L.** Chúng chỉ khác nhau ở thứ tự các việc **cùng hạn chót**; nhóm việc cùng hạn d luôn chiếm cùng một khoảng thời gian, và việc cuối nhóm luôn kết thúc cùng thời điểm, nên độ trễ lớn nhất của nhóm không đổi.
3. **Có lịch tối ưu không nghịch thế.** Nếu lịch tối ưu O có nghịch thế, thì có một nghịch thế **kề nhau**: i làm ngay trước j với $d_j < d_i$. Đổi chỗ i và j:
   a. j xong sớm hơn, độ trễ của j không tăng.
   b. i giờ xong đúng lúc j xong trước đây (gọi là f). Độ trễ mới của i là $f - d_i < f - d_j$ = độ trễ cũ của j.
   c. Vậy L không tăng, và số nghịch thế giảm 1. Lặp lại tối đa $n(n-1)/2$ lần.
4. **Kết luận:** lịch tối ưu không nghịch thế, không rỗi có cùng L với EDF (bước 2), nên EDF tối ưu.

Hình ảnh: hãy tưởng tượng một hàng người xếp hàng sai thứ tự hạn chót. Mỗi lần đổi chỗ hai người đứng cạnh nhau sai thứ tự, hàng không tệ đi; đổi đủ lần, hàng trở thành đúng thứ tự EDF.

Độ phức tạp: sắp xếp $O(n \log n)$, xếp lịch $O(n)$.

4.3 Bộ nhớ đệm tối ưu: lập luận trao đổi phức tạp hơn

Khi biết trước toàn bộ chuỗi yêu cầu, chiến lược **loại phần tử sẽ được dùng lại xa nhất trong tương lai** (Farthest-in-Future, thuật toán của Belady) cho số lần trượt bộ nhớ đệm ít nhất.

Bối cảnh và phát biểu

Máy tính có bộ nhớ nhỏ nhưng nhanh (cache) và bộ nhớ lớn nhưng chậm. Cache chứa được k phần tử. Có chuỗi yêu cầu $d_1, d_2, \ldots, d_m$. Nếu phần tử được yêu cầu đã có trong cache: **trúng** (hit). Nếu chưa có: **trượt** (miss), phải nạp nó vào; nếu cache đầy thì phải **loại** một phần tử. Mục tiêu: chọn phần tử để loại sao cho tổng số lần trượt nhỏ nhất.

Với người quản trị hệ thống, đây là bài toán quen thuộc: bộ đệm trang của hệ điều hành, cache của trình duyệt, cache của máy chủ web — tất cả đều phải quyết định "đuổi ai ra" khi đầy.

Hình ảnh: một bàn làm việc chỉ để được k cuốn sách, phần còn lại nằm trên giá ở phòng bên. Khi cần cuốn không có trên bàn, phải sang phòng bên lấy và cất bớt một cuốn đang có.

Quy tắc Farthest-in-Future

Khi phải loại, chọn phần tử trong cache có **lần yêu cầu tiếp theo xa nhất** (hoặc không bao giờ được yêu cầu nữa).

Ví dụ: k = 2, cache ban đầu {a, b}, chuỗi yêu cầu a, b, c, b, c, a, b.

Bước	Yêu cầu	Trạng thái	Quyết định	Cache sau
1	a	trúng	—	{a, b}
2	b	trúng	—	{a, b}
3	c	**trượt**	a dùng lại ở bước 6, b ở bước 4 → loại a	{b, c}
4	b	trúng	—	{b, c}
5	c	trúng	—	{b, c}
6	a	**trượt**	b dùng lại ở bước 7, c không bao giờ → loại c	{a, b}
7	b	trúng	—	{a, b}

Tổng cộng 2 lần trượt. Nếu ở bước 3 loại b thay vì a, thì bước 4 trượt ngay.

Chứng minh (phác thảo theo sách)

Lịch rút gọn. Một lịch gọi là *rút gọn* nếu chỉ nạp phần tử vào cache khi nó được yêu cầu và đang trượt. Mọi lịch đều có thể biến thành lịch rút gọn mà số lần nạp không tăng ("trì hoãn việc nạp đến đúng lúc cần"). Vì vậy chỉ cần so sánh với lịch rút gọn.

Bổ đề trao đổi. Giả sử lịch rút gọn S trùng với lịch Farthest-in-Future S_FF ở j yêu cầu đầu tiên. Khi đó tồn tại lịch rút gọn S′ trùng với S_FF ở j + 1 yêu cầu đầu và có số lần trượt **không nhiều hơn** S.

Tại yêu cầu j + 1 với phần tử d, chỉ khác biệt thực sự khi cả hai đều trượt nhưng S_FF loại e còn S loại f ≠ e. Ta cho S′ loại e (như S_FF), sau đó S′ bắt chước S cho đến khi hai cache trùng lại. Điểm then chốt: vì S_FF chọn e là phần tử **xa nhất**, nên f sẽ được yêu cầu **trước** e. Xét yêu cầu đầu tiên chạm tới e hoặc f, hoặc buộc S phải loại e:

1. S loại e để nạp một phần tử g khác: S′ loại f thay vào đó — hai cache trùng nhau.
2. Yêu cầu f đến: S trượt (vì đã loại f), S′ trúng (vì giữ f). S′ chỉ có thể tốt hơn; sau khi điều chỉnh để S′ vẫn là lịch rút gọn, hai cache trùng nhau.
3. Yêu cầu e đến trước f: **không thể xảy ra**, vì e là phần tử được dùng lại xa nhất.

Áp dụng bổ đề lặp lại từ j = 0 đến m, một lịch tối ưu được biến dần thành S_FF mà số lần trượt không tăng. Vậy Farthest-in-Future tối ưu.

Từ ngoại tuyến đến trực tuyến

Trên máy tính thật, ta **không biết tương lai**. Chính sách phổ biến là LRU (Least Recently Used): loại phần tử lâu nhất không được dùng — tức là dùng quá khứ để "đoán" tương lai theo nguyên lý cục bộ. Farthest-in-Future vẫn rất có giá trị: nó là **chuẩn so sánh** để đánh giá mọi chính sách trực tuyến. Sách quay lại bài toán trực tuyến này bằng thuật toán ngẫu nhiên ở Chương 13.

4.4 Đường đi ngắn nhất trong đồ thị: thuật toán Dijkstra

Với đồ thị có trọng số **không âm**, Dijkstra tìm đường đi ngắn nhất từ một đỉnh nguồn s tới mọi đỉnh bằng cách mỗi bước "chiếm" đỉnh chưa xét **gần s nhất**; với hàng đợi ưu tiên, thời gian là O(m log n).

Phát biểu

Đồ thị có hướng G = (V, E), n đỉnh, m cạnh; mỗi cạnh e có độ dài l_e ⩾ 0. Cho đỉnh nguồn s, tìm độ dài d(v) của đường đi ngắn nhất từ s tới mọi đỉnh v, cùng chính các đường đi đó.

Hình ảnh: đổ nước vào một hệ thống kênh tại điểm s. Nước loang ra theo mọi kênh với cùng tốc độ; thời điểm nước **chạm tới** mỗi điểm chính là khoảng cách ngắn nhất tới điểm đó. Dijkstra mô phỏng chính xác làn nước này: mỗi bước ghi nhận điểm tiếp theo mà nước chạm tới.

Thuật toán

Duy trì tập S các đỉnh đã "chốt" khoảng cách, ban đầu S = {s}, d(s) = 0. Mỗi bước:

```
Với mỗi đỉnh v ∉ S, tính
    d'(v) = min { d(u) + l_(u,v) :  u ∈ S, (u, v) ∈ E }
Chọn v ∉ S có d'(v) nhỏ nhất
Thêm v vào S, đặt d(v) = d'(v), ghi nhớ u làm "cha" của v
```

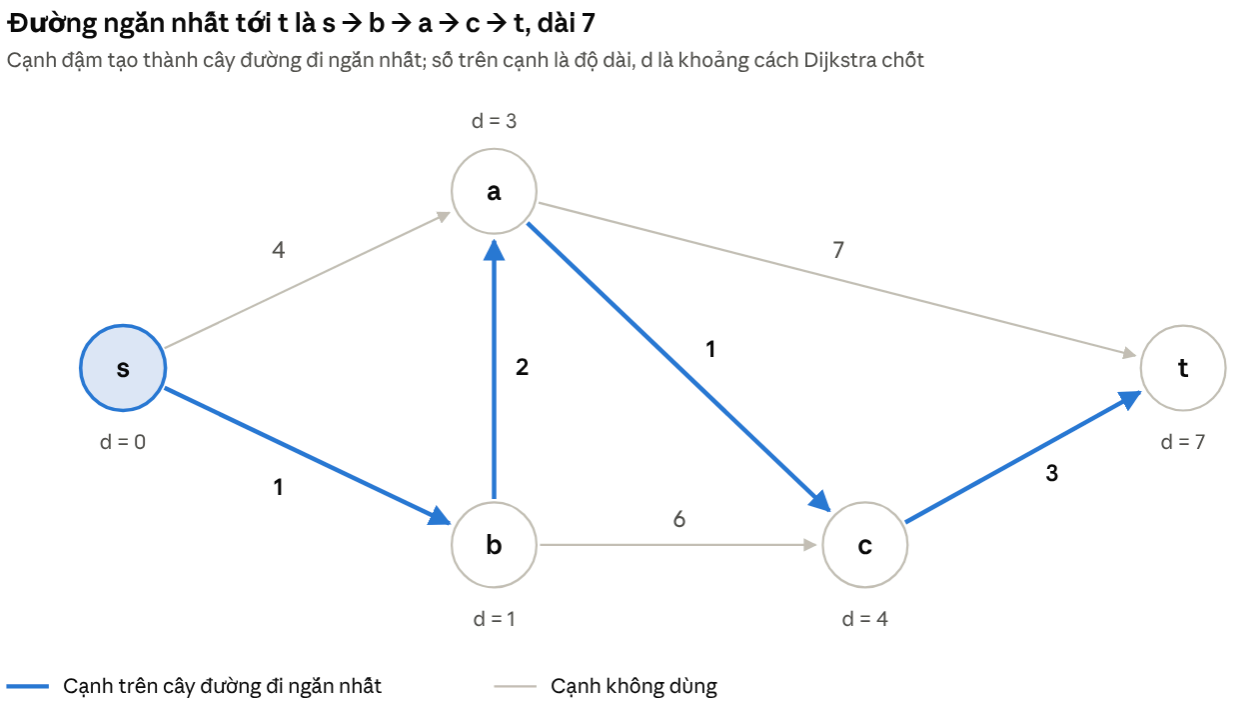
Ví dụ chạy tay

Các cạnh: s→a 4, s→b 1, b→a 2, b→c 6, a→c 1, a→t 7, c→t 3.

Lượt	Tập S sau lượt	Đỉnh được chốt	d′(a)	d′(b)	d′(c)	d′(t)
0	{s}	s (0)	4	**1**	∞	∞
1	{s, b}	b (1)	**3**	—	7	∞
2	{s, b, a}	a (3)	—	—	**4**	10
3	{s, b, a, c}	c (4)	—	—	—	**7**

Lượt	Tập S sau lượt	Đỉnh được chốt	d′(a)	d′(b)	d′(c)	d′(t)
4	{s, b, a, c, t}	t (7)	—	—	—	—

Đường đi ngắn nhất tới t: s → b → a → c → t, độ dài 1 + 2 + 1 + 3 = 7. Lưu ý đường "trực tiếp" s → a có vẻ ngắn (4) nhưng đi vòng qua b lại chỉ tốn 3.



Hình 4.4 · đồ thị ví dụ và cây đường đi ngắn nhất

Các cạnh đậm là cạnh "cha" mà Dijkstra ghi nhớ khi chốt mỗi đỉnh; chúng tạo thành một cây gốc s chứa đường ngắn nhất tới mọi đỉnh. Cạnh s → a (4) và a → t (7) bị bỏ vì có đường vòng ngắn hơn.

Chứng minh: tham lam luôn đi trước

Bất biến. Tại mọi thời điểm, với mỗi u ∈ S, đường đi P_u mà thuật toán tìm được là đường ngắn nhất từ s tới u.

Chứng minh quy nạp theo |S|. Giả sử bất biến đúng khi |S| = k, thuật toán thêm v qua cạnh (u, v). Xét một đường P bất kỳ từ s tới v. P bắt đầu trong S và phải **ra khỏi S** tại cạnh đầu tiên (x, y) với x ∈ S, y ∉ S. Khi đó:

```
độ dài(P) ≥ d(x) + l_(x,y) ≥ d'(y) ≥ d'(v) = độ dài(P_v)
```

Bất đẳng thức đầu dùng giả thiết l_e ≥ 0 (phần còn lại của P sau y không âm); bất đẳng thức cuối đúng vì thuật toán chọn v có d′(v) nhỏ nhất. Vậy P_v ngắn nhất.

Cạnh âm phá vỡ thuật toán. Nếu có cạnh âm, phần đường "đi vòng" sau y có thể làm tổng giảm, nên bất đẳng thức đầu sai. Bài toán có cạnh âm cần thuật toán Bellman–Ford (Chương 6).

Cài đặt với hàng đợi ưu tiên

Lưu các đỉnh v ∉ S trong một heap với khoá d′(v). Mỗi lượt ExtractMin lấy đỉnh kế tiếp (n lần); khi chốt v, cập nhật d′(w) cho các cạnh (v, w) bằng ChangeKey (tổng cộng tối đa m lần). Mỗi thao tác heap tốn O(log n), nên tổng thời gian **O(m log n)**.

```
import heapq

def dijkstra(graph, s):
    """graph: {u: [(v, l_uv), ...]}, l_uv >= 0. Trả về (khoảng cách, cha)."""
    dist, parent = {s: 0}, {s: None}
    done, heap = set(), [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if u in done:                    # bản ghi cũ (thay cho ChangeKey)
            continue
        done.add(u)
        for v, l in graph.get(u, []):
            if v not in done and d + l < dist.get(v, float('inf')):
                dist[v], parent[v] = d + l, u
                heapq.heappush(heap, (d + l, v))
    return dist, parent

g = {'s': [('a', 4), ('b', 1)], 'b': [('a', 2), ('c', 6)],
     'a': [('c', 1), ('t', 7)], 'c': [('t', 3)]}
print(dijkstra(g, 's')[0])
# {'s': 0, 'a': 3, 'b': 1, 'c': 4, 't': 7}
```

4.5 Cây khung nhỏ nhất

Ba thuật toán tham lam khác nhau — Kruskal, Prim, Reverse-Delete — đều cho cây khung nhỏ nhất, và cả ba được chứng minh bằng chỉ hai tính chất: **tính chất lát cắt** và **tính chất chu trình**.

Phát biểu

Đồ thị vô hướng liên thông G = (V, E), mỗi cạnh e có chi phí c_e > 0. Tìm tập cạnh T ⊆ E sao cho đồ thị (V, T) liên thông và tổng chi phí nhỏ nhất. Lời giải luôn là một **cây** (nếu có chu trình, bỏ một cạnh trên chu trình vẫn liên thông và rẻ hơn), gọi là cây khung nhỏ nhất (MST).

Hình ảnh: cần nối n trạm bơm hoặc n điểm đo bằng cáp truyền dữ liệu, mỗi tuyến cáp có chi phí; muốn mọi trạm thông được với nhau với tổng chi phí cáp ít nhất.

Ba thuật toán

Thuật toán	Quy tắc	Hình ảnh
Kruskal	Duyệt cạnh theo chi phí tăng dần; thêm cạnh nếu không tạo chu trình	Nhiều "hòn đảo" nhỏ, nối dần bằng cây cầu rẻ nhất chưa thừa
Prim	Bắt đầu từ một đỉnh; mỗi bước thêm cạnh rẻ nhất nối cây hiện tại với đỉnh ngoài cây	Một vết dầu loang, mỗi bước lan sang đỉnh gần nhất (giống Dijkstra nhưng đo cạnh, không đo đường)
Reverse-Delete	Duyệt cạnh theo chi phí giảm dần; xoá cạnh nếu xoá xong vẫn liên thông	Cắt tỉa: bỏ các tuyến đắt nhất miễn là mạng không đứt

Ví dụ chạy tay Kruskal

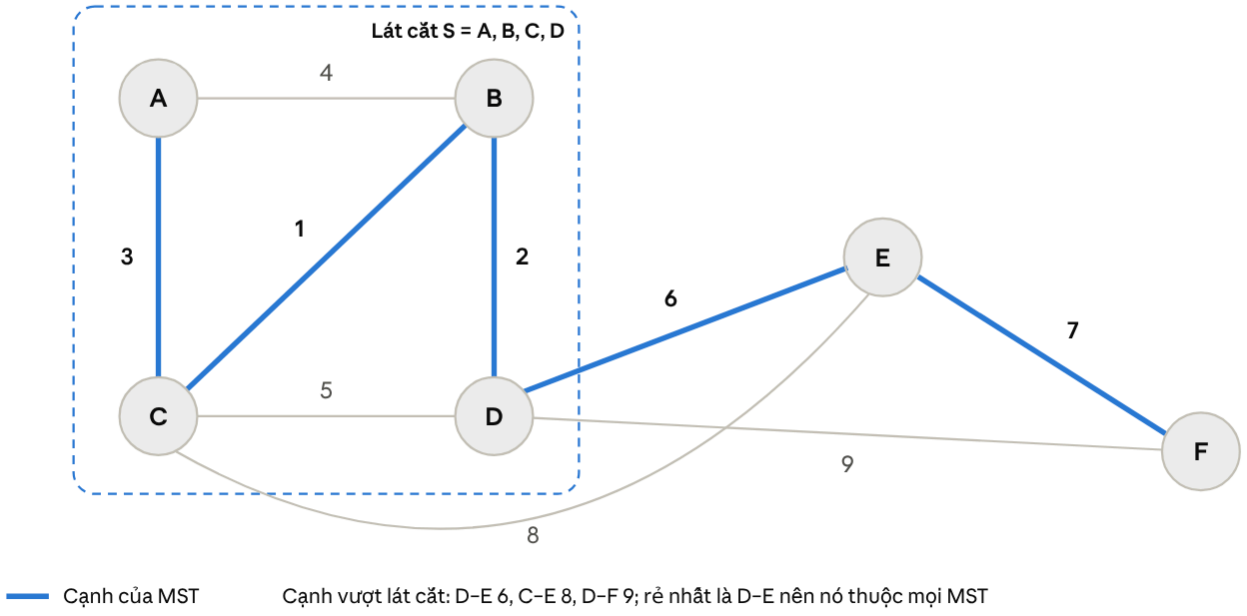
Đồ thị 6 đỉnh A–F, các cạnh theo chi phí tăng dần: B–C 1, B–D 2, A–C 3, A–B 4, C–D 5, D–E 6, E–F 7, C–E 8, D–F 9.

Cạnh	Chi phí	Quyết định	Lý do
B–C	1	thêm	nối hai đảo {B}, {C}
B–D	2	thêm	
A–C	3	thêm	
A–B	4	bỏ	A, B đã cùng đảo → tạo chu trình A–C–B
C–D	5	bỏ	tạo chu trình C–B–D
D–E	6	thêm	
E–F	7	thêm	đã đủ n − 1 = 5 cạnh, dừng

Cây khung nhỏ nhất có tổng chi phí 1 + 2 + 3 + 6 + 7 = **19**. Chạy Prim từ A cho đúng cùng cây: A–C, C–B, B–D, D–E, E–F.

Cây khung nhỏ nhất có tổng chi phí 19

Cạnh đậm thuộc MST; khung đứt nét là một lát cắt minh hoạ tính chất lát cắt



Hình 4.5 · cây khung nhỏ nhất và một lát cắt

Hãy thử vẽ một lát cắt bất kỳ khác (ví dụ S = {A}): cạnh rẻ nhất vượt qua nó (A–C, chi phí 3) cũng luôn nằm trên cây đậm.

Hai tính chất then chốt (giả sử mọi chi phí khác nhau)

Tính chất lát cắt. Với mọi tập S khác rỗng và khác V, cạnh e rẻ nhất có một đầu trong S, một đầu trong V − S **thuộc mọi** cây khung nhỏ nhất.

Chứng minh (trao đổi). Giả sử cây khung T không chứa e = (v, w). Trong T có đường đi từ v tới w; đường này phải vượt lát cắt tại một cạnh e′ ≠ e. Thay e′ bằng e: T′ = T − e′ + e vẫn là cây khung, và vì c_e < c_e′ nên T′ rẻ hơn T. Vậy T không thể nhỏ nhất.

Lưu ý tinh tế: phải chọn e′ **trên đường đi từ v tới w**, không phải một cạnh vượt lát cắt bất kỳ — nếu không T′ có thể không liên thông.

Tính chất chu trình. Trên mọi chu trình C, cạnh **đắt nhất** không thuộc cây khung nhỏ nhất nào. Chứng minh tương tự bằng trao đổi.

Vì sao cả ba thuật toán đúng

- **Kruskal:** khi thêm cạnh (v, w), lấy S là tập đỉnh đang cùng đảo với v. Đây là cạnh rẻ nhất vượt lát cắt S (mọi cạnh rẻ hơn đã xét và nằm trong các đảo) → đúng theo tính chất lát cắt.
- **Prim:** S là tập đỉnh của cây hiện tại; cạnh được thêm chính là cạnh rẻ nhất vượt S → đúng theo tính chất lát cắt.

- **Reverse-Delete:** cạnh bị xoá là cạnh đắt nhất trên một chu trình trong đồ thị còn lại → đúng theo tính chất chu trình.

Nếu có các chi phí bằng nhau, ta "nhiễu" chi phí một lượng rất nhỏ để chúng khác nhau; các kết luận vẫn đúng.

Độ phức tạp

Prim cài đặt với heap giống Dijkstra: **O(m log n)**. Kruskal cần sắp xếp cạnh O(m log m) = O(m log n) và một cấu trúc để kiểm tra nhanh "hai đỉnh có cùng đảo không" — đó là nội dung của mục 4.6.

4.6 Cài đặt Kruskal: cấu trúc dữ liệu Union-Find

Union-Find quản lý một họ các tập rời nhau và trả lời nhanh câu hỏi "hai phần tử có cùng tập không"; với quy tắc **gộp tập nhỏ vào tập lớn**, mỗi Find tốn O(log n), giúp Kruskal chạy trong O(m log n).

Ba thao tác

Thao tác	Ý nghĩa	Trong Kruskal
MakeUnionFind(S)	Tạo mỗi phần tử thành một tập riêng	Ban đầu mỗi đỉnh là một đảo
Find(u)	Trả về "tên" của tập chứa u	Kiểm tra Find(v) = Find(w)? Nếu bằng, cạnh (v, w) tạo chu trình
Union(A, B)	Gộp hai tập thành một	Thêm cạnh thì gộp hai đảo

Kruskal thực hiện tối đa 2m lần Find và n − 1 lần Union.

Hình ảnh: mỗi tập là một "câu lạc bộ" có một chủ tịch; Find(u) là hỏi "chủ tịch câu lạc bộ của u là ai"; Union là sáp nhập hai câu lạc bộ.

Cách 1: dùng mảng

Mảng Component[u] lưu tên tập của u.

- Find(u): đọc Component[u], **O(1)**.
- Union(A, B): phải sửa tên mọi phần tử của một tập, có thể tới O(n).
- Cải tiến: luôn giữ tên của **tập lớn hơn**, chỉ sửa tên các phần tử của tập nhỏ. Mỗi lần tên của phần tử v bị sửa, tập chứa v ít nhất **gấp đôi**. Tập không thể lớn hơn n, nên tên của v bị sửa tối đa $\log_2 n$ lần. Do đó k lần Union tốn tổng cộng O(k log k).

Với cách này, Kruskal tốn O(m) cho các lần Find và O(n log n) cho các lần Union.

Cách 2: dùng con trỏ (cây)

Mỗi tập là một cây; mỗi phần tử trỏ tới "cha"; gốc cây là tên tập.

- Union(A, B): cho gốc của tập **nhỏ hơn** trỏ vào gốc tập lớn hơn: **O(1)**.
- Find(u): đi theo con trỏ từ u lên gốc. Mỗi lần đường từ u lên gốc dài thêm 1, tập chứa u ít nhất gấp đôi, nên độ sâu ⩽ $\log_2 n$ và Find tốn **O(log n)**.

```
Find(u):    while parent[u] ≠ u:  u ← parent[u]
            return u
Union(a, b):   // a, b là gốc
            if size[a] < size[b]: đổi chỗ a, b
            parent[b] ← a;   size[a] ← size[a] + size[b]
```

Cải tiến: nén đường đi (path compression)

Sau mỗi Find(u), cho mọi đỉnh trên đường đi từ u lên gốc **trỏ thẳng vào gốc**. Các lần Find sau trên những đỉnh này chỉ tốn một bước. Kết hợp với gộp theo kích thước, tổng chi phí của dãy thao tác gần như tuyến tính: O(α(n)) mỗi thao tác, trong đó α là hàm Ackermann ngược, tăng chậm đến mức α(n) ⩽ 4 với mọi n thực tế. Hình ảnh: lần đầu hỏi đường lên gặp chủ tịch phải qua nhiều cấp; sau đó, ai đã đi qua đều được cho số điện thoại trực tiếp của chủ tịch.

Cài đặt Kruskal hoàn chỉnh

```
def kruskal(n, edges):
    """n đỉnh 0..n-1; edges: danh sách (c, u, v). Trả về (tổng chi phí, các
cạnh của MST)."""
    parent, size = list(range(n)), [1] * n

    def find(u):
        root = u
        while parent[root] != root:
            root = parent[root]
        while parent[u] != root:          # nén đường đi
            parent[u], u = root, parent[u]
        return root

    total, tree = 0, []
    for c, u, v in sorted(edges):         # O(m log m)
        ru, rv = find(u), find(v)
        if ru != rv:                      # khác đảo -> không tạo chu trình
            if size[ru] < size[rv]:
```

```
                ru, rv = rv, ru
            parent[rv] = ru; size[ru] += size[rv]
            total += c; tree.append((u, v, c))
    return total, tree

A, B, C, D, E, F = range(6)
edges = [(1,B,C),(2,B,D),(3,A,C),(4,A,B),(5,C,D),(6,D,E),(7,E,F),(8,C,E),
(9,D,F)]
print(kruskal(6, edges)[0])   # 19
```

4.7 Phân cụm: Kruskal dừng sớm

Để chia n đối tượng thành k cụm sao cho hai cụm khác nhau **cách nhau xa nhất có thể**, chỉ cần chạy Kruskal và dừng khi còn đúng k thành phần — tương đương xoá k − 1 cạnh đắt nhất của cây khung nhỏ nhất.

Phát biểu

Cho tập U gồm n đối tượng p_1, …, p_n và hàm khoảng cách d(p_i, p_j) ⩾ 0, đối xứng, d(p_i, p_i) = 0. Một **k-phân cụm** chia U thành k nhóm khác rỗng. **Độ giãn cách** (spacing) của một phân cụm là khoảng cách nhỏ nhất giữa hai điểm thuộc hai cụm khác nhau:

```
spacing(C) = min { d(p, q) :  p, q thuộc hai cụm khác nhau của C }
```

Mục tiêu: tìm k-phân cụm có độ giãn cách **lớn nhất**, tức là các cụm tách biệt rõ nhất.

Hình ảnh: n trạm quan trắc mưa trên một lưu vực, khoảng cách là mức khác biệt giữa chuỗi số liệu của hai trạm. Muốn chia thành k vùng khí hậu sao cho hai trạm ở hai vùng khác nhau luôn khác biệt ít nhất một ngưỡng càng lớn càng tốt.

Thuật toán (single-linkage)

```
Coi mỗi điểm là một cụm; xét các cặp điểm theo khoảng cách tăng dần
   nếu hai điểm thuộc hai cụm khác nhau: gộp hai cụm (thêm "cạnh")
   dừng khi còn đúng k cụm
```

Đây chính là Kruskal trên đồ thị đầy đủ với chi phí cạnh là khoảng cách, chỉ khác là **dừng trước k − 1 cạnh cuối**.

Ví dụ với đồ thị ở mục 4.5 và k = 3: Kruskal thêm B–C (1), B–D (2), A–C (3) và lúc này còn 3 cụm {A, B, C, D}, {E}, {F} — dừng. Độ giãn cách là cạnh rẻ nhất nối hai cụm khác nhau: min(D–E 6, E–F 7, C–E 8, D–F 9) = **6**, đúng bằng cạnh mà Kruskal định thêm tiếp theo.

Chứng minh

Gọi C* là phân cụm thuật toán tìm được, d* là chi phí cạnh tiếp theo Kruskal **sẽ** thêm nếu chạy tiếp. Khi đó spacing(C*) = d*. Xét một k-phân cụm C khác bất kỳ. Vì C ≠ C*, có hai điểm p_i, p_j **cùng cụm** trong C* nhưng **khác cụm** trong C.

1. Trong C*, p_i và p_j được nối bởi một đường gồm các cạnh mà Kruskal đã thêm — mỗi cạnh có chi phí ⩽ d*.
2. Vì p_i và p_j ở hai cụm khác nhau của C, đường đó phải có một cạnh (p, q) mà p, q thuộc hai cụm khác nhau của C.
3. Do đó spacing(C) ⩽ d(p, q) ⩽ d* = spacing(C*).

Vậy không phân cụm nào tốt hơn C*. Đây là một lập luận kiểu "vật cản": mọi phân cụm khác buộc phải cắt ngang một cạnh ngắn.

Độ phức tạp: O(m log n) như Kruskal (với n điểm, đồ thị đầy đủ có m = n(n − 1)/2 cặp).

4.8 Mã Huffman và nén dữ liệu

Để mã hoá văn bản bằng ít bit nhất, Huffman **gộp lặp lại hai ký tự hiếm nhất** thành một "ký tự gộp"; cây nhị phân thu được cho mã tiền tố tối ưu.

Từ mã độ dài cố định đến mã tiền tố

Với 32 ký hiệu, mã độ dài cố định cần 5 bit mỗi ký hiệu. Nhưng các chữ cái xuất hiện với tần suất rất khác nhau (trong tiếng Anh, "e" nhiều hơn "z" hàng chục lần), nên gán mã **ngắn cho ký tự phổ biến**, dài cho ký tự hiếm sẽ tiết kiệm hơn — ý tưởng đã có từ mã Morse.

Morse gặp vấn đề **mơ hồ**: e = "·", t = "–", a = "·–", nên chuỗi "·–" có thể là "a" hoặc "et"; Morse phải chèn khoảng lặng giữa các chữ. Lời giải là **mã tiền tố**: không mã của ký tự nào là phần đầu của mã ký tự khác. Khi đó đọc bit từ trái sang phải, ngay khi khớp một mã là chắc chắn đã đọc xong một ký tự.

Đo chất lượng: độ dài bit trung bình

Với bảng chữ cái S, tần suất f_x (tổng bằng 1), mã γ:

```
ABL(γ) = Σ  f_x · |γ(x)|        (tổng theo mọi ký tự x thuộc S)
```

ABL là số bit trung bình mỗi ký tự. Mục tiêu: tìm mã tiền tố có ABL nhỏ nhất.

Mã tiền tố = cây nhị phân

Mỗi mã tiền tố ứng với một cây nhị phân có các ký tự ở **lá**: đi xuống nhánh trái ghi 0, nhánh phải ghi 1; mã của ký tự là dãy bit trên đường từ gốc đến lá của nó. Vì ký tự nằm ở lá, không đường nào là tiền tố của đường khác. Khi đó $|\gamma(x)|$ = độ sâu của lá x, và bài toán trở thành: **tìm cây nhị phân làm nhỏ $\Sigma f_x \cdot$ độ sâu(x)**.

Ba nhận xét dẫn tới thuật toán:

1. Cây tối ưu là cây **đầy đủ**: mọi nút trong có đúng hai con (nếu một nút chỉ có một con, kéo con lên sẽ rút ngắn mã).
2. Ký tự tần suất thấp nên năm sâu hơn: nếu $f_y < f_z$ mà y nông hơn z, đổi chỗ y và z làm ABL giảm (lập luận trao đổi).
3. Do đó có một cây tối ưu trong đó **hai ký tự hiếm nhất là anh em** ở tầng sâu nhất.

Thuật toán Huffman

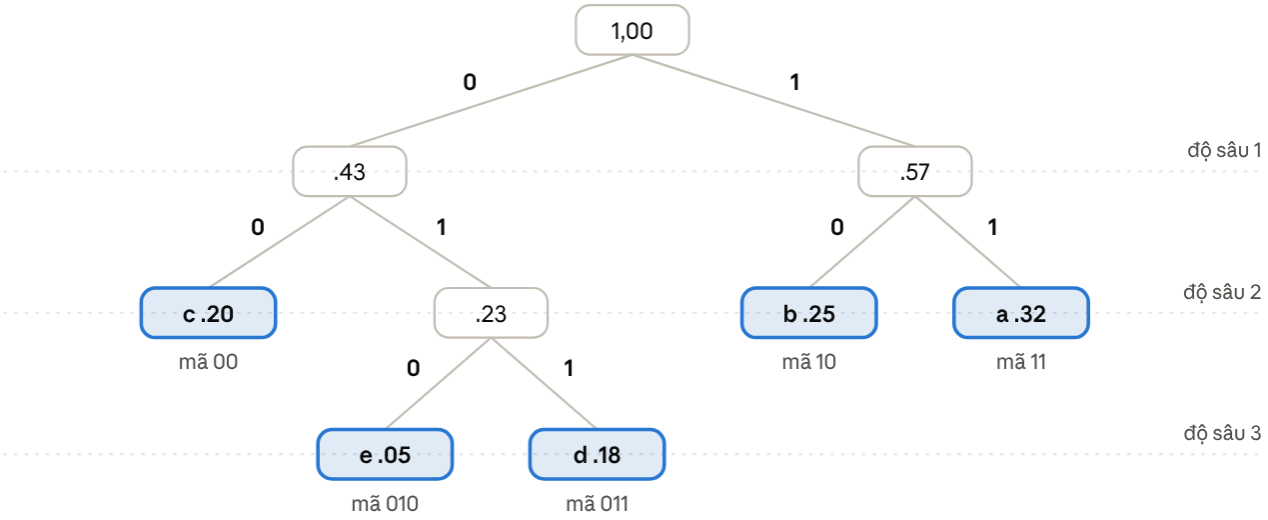
```
Huffman(S):
   nếu |S| = 2: gán 0 cho một ký tự, 1 cho ký tự kia
   ngược lại:
      lấy hai ký tự y*, z* có tần suất nhỏ nhất
      thay chúng bằng ký tự gộp ω với f_ω = f_y* + f_z*
      đệ quy Huffman(S − {y*, z*} ∪ {ω}) được cây T'
      tách lá ω thành nút có hai con y*, z*
```

Ví dụ (như trong sách): a .32, b .25, c .20, d .18, e .05.

Bước	Hai ký tự hiếm nhất	Ký tự gộp	Tập còn lại
1	e .05, d .18	(de) .23	a .32, b .25, c .20, (de) .23
2	c .20, (de) .23	(cde) .43	a .32, b .25, (cde) .43
3	b .25, a .32	(ab) .57	(cde) .43, (ab) .57
4	(cde) .43, (ab) .57	gốc 1.00	—

Ký tự hiếm nằm sâu hơn: d và e dài 3 bit, các ký tự còn lại 2 bit

Cây Huffman cho a .32, b .25, c .20, d .18, e .05; nhánh trái ghi 0, nhánh phải ghi 1; ABL = 2,23 bit



Thứ tự gộp: e + d → .23, c + .23 → .43, b + a → .57, .43 + .57 → gốc

Hình 4.8 · cây Huffman và mã của từng ký tự

Đọc mã của một ký tự bằng cách đi từ gốc xuống lá của nó và ghi lại các bit trên đường. Vì mọi ký tự đều nằm ở lá, không mã nào là phần đầu của mã khác.

Mã thu được: a = 11, b = 10, c = 00, e = 010, d = 011. Khi đó:

```
ABL = 0,32·2 + 0,25·2 + 0,20·2 + 0,18·3 + 0,05·3 = 2,23 bit
```

so với 3 bit của mã độ dài cố định cho 5 ký tự — tiết kiệm khoảng 26%.

Chứng minh tối ưu (quy nạp theo số ký tự)

Điểm then chốt là đẳng thức liên hệ cây T (có y*, z*) và cây T′ (có ω thay cho cặp đó):

```
ABL(T) = ABL(T') + f_ω
```

vì y*, z* sâu hơn ω một tầng, nên mỗi lần dùng y* hoặc z* tốn thêm đúng 1 bit. Giả sử có cây Z tốt hơn T. Theo nhận xét 3, có thể giả sử y*, z* là anh em trong Z. Gộp chúng thành ω được Z′ với ABL(Z′) = ABL(Z) − f_ω < ABL(T) − f_ω = ABL(T′), mâu thuẫn với giả thiết quy nạp rằng T′ tối ưu.

Cài đặt

Dùng hàng đợi ưu tiên khoá theo tần suất: k − 1 lần lấy hai phần tử nhỏ nhất và chèn một phần tử gộp, mỗi lần O(log k), tổng **O(k log k)** với k ký tự.

```
import heapq, itertools

def huffman(freq):
    """freq: {ký tự: tần suất}. Trả về {ký tự: mã nhị phân}."""
    tie = itertools.count()                       # phá thế hoà khi tần suất
bằng nhau
    heap = [(f, next(tie), ch) for ch, f in freq.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        f1, _, left = heapq.heappop(heap)          # hai ký tự hiếm nhất
        f2, _, right = heapq.heappop(heap)
        heapq.heappush(heap, (f1 + f2, next(tie), (left, right)))
    code = {}
    def walk(node, prefix):
        if isinstance(node, tuple):
            walk(node[0], prefix + '0'); walk(node[1], prefix + '1')
        else:
            code[node] = prefix or '0'
    walk(heap[0][2], '')
    return code

print(huffman({'a': .32, 'b': .25, 'c': .20, 'd': .18, 'e': .05}))
# {'c': '00', 'e': '010', 'd': '011', 'b': '10', 'a': '11'}
```

Lưu ý: mã cụ thể của d và e có thể đổi chỗ cho nhau tuỳ cách gán 0/1, nhưng độ dài mã và ABL = 2,23 không đổi. Huffman là nền tảng của nhiều định dạng nén thực tế (ví dụ DEFLATE trong ZIP/gzip, một bước trong JPEG).

4.9 Cây khung có hướng chi phí nhỏ nhất: tham lam nhiều pha

Trên đồ thị **có hướng**, chọn cạnh vào rẻ nhất cho mỗi đỉnh có thể tạo chu trình; thuật toán Chu–Liu/Edmonds xử lý bằng cách **trừ chi phí** rồi **co chu trình thành một đỉnh** và lặp lại, cho lời giải tối ưu trong O(mn).

Phát biểu

Đồ thị có hướng G = (V, E), một đỉnh gốc r, mỗi cạnh có chi phí c_e ⩾ 0. Một **cây khung có hướng** (arborescence) gốc r là tập cạnh T sao cho với mọi đỉnh v, có **đúng một đường đi** từ r tới v trong T. Tương đương:

- mỗi đỉnh v ≠ r có **đúng một cạnh đi vào** trong T, và
- T không chứa chu trình.

Tìm cây khung có hướng có tổng chi phí nhỏ nhất. Hình ảnh: một mạng phát tin (hay mạng cấp nước một chiều) từ trạm gốc r; mỗi trạm chỉ cần nhận từ đúng một trạm khác, và kênh chỉ chảy theo một chiều.

Vì sao tham lam đơn giản thất bại

Nếu mỗi đỉnh v ≠ r chỉ chọn cạnh vào rẻ nhất, ta có đúng một cạnh vào mỗi đỉnh — nhưng các cạnh đó có thể tạo **chu trình** không nối được tới r. Các thuật toán MST vô hướng (Kruskal, Prim) cũng không áp dụng trực tiếp.

Ý tưởng 1: điều chỉnh chi phí

Với mỗi v ≠ r, gọi y_v là chi phí cạnh vào rẻ nhất của v. Định nghĩa chi phí mới cho mọi cạnh e = (u, v):

```
c'_e = c_e − y_v   (≥ 0)
```

Vì **mọi** cây khung có hướng có đúng một cạnh vào mỗi đỉnh v ≠ r, ta có cost(T) = cost′(T) + Σ y_v với mọi T. Hằng số Σ y_v như nhau cho mọi T, nên **T tối ưu theo c khi và chỉ khi tối ưu theo c′**. Sau điều chỉnh, mỗi đỉnh có ít nhất một cạnh vào chi phí 0.

Ý tưởng 2: co chu trình

Gọi F* là tập các cạnh vào chi phí 0 (mỗi đỉnh chọn một). Nếu F* là cây khung có hướng thì nó có chi phí c′ bằng 0, chắc chắn tối ưu. Nếu không, F* chứa một chu trình C toàn cạnh chi phí 0. Ta **co C thành một siêu đỉnh**, giải đệ quy trên đồ thị nhỏ hơn, rồi **mở** C ra: giữ mọi cạnh của C trừ cạnh đi vào đỉnh mà lời giải đệ quy đã đi vào C. Mấu chốt chứng minh: tồn tại lời giải tối ưu đi vào C **đúng một lần**, nên việc co không làm mất tối ưu.

Ví dụ chạy tay

Đồ thị gốc r với các cạnh: r→a 6, r→b 8, a→b 2, b→a 1, b→c 3, c→a 4.

Pha	Cạnh vào rẻ nhất (y)	Chi phí sau khi trừ	Cạnh chi phí 0	Chu trình?
1	a: b→a 1; b: a→b 2; c: b→c 3	r→a 5, r→b 6, c→a 3, các cạnh còn lại 0	b→a, a→b, b→c	a ↔ b → co thành X
2	X: c→a 3; c: X→c 0	r→a 2, r→b 3, c→a 0	c→X, X→c	X ↔ c → co thành Y
3	Y: r→a 2	r→a 0	r→Y	không — dừng

Mở ngược lại. Ở pha 3, Y được vào qua r→a, tức vào X tại đỉnh a. Mở Y (chu trình X ↔ c): bỏ cạnh c→X, giữ X→c, tức b→c. Mở X (chu trình a ↔ b) với a là điểm vào: bỏ b→a, giữ a→b. Kết quả:

```
T = { r→a, a→b, b→c },   chi phí = 6 + 2 + 3 = 11
```

Kiểm tra: tổng các lượng đã trừ 1 + 2 + 3 + 3 + 2 = 11, khớp với chi phí T (vì chi phí điều chỉnh cuối cùng bằng 0). Phương án khác r→b, b→a, b→c tốn 8 + 1 + 3 = 12, đắt hơn.

Độ phức tạp

Mỗi pha (tìm cạnh vào rẻ nhất, trừ chi phí, tìm chu trình, co) tốn O(m); mỗi lần co giảm số đỉnh ít nhất 1, nên tối đa n pha: **O(mn)**. Có các cài đặt nhanh hơn, nhưng sách dừng ở mức này. Bài học của mục: đôi khi một bước tham lam đơn thuần không đủ, nhưng tham lam **kết hợp biến đổi bài toán** (điều chỉnh chi phí, co đỉnh) vẫn cho lời giải tối ưu.

Tổng kết: khi nào tham lam đúng

Tham lam đúng khi ta chứng minh được rằng mỗi lựa chọn cục bộ **không bao giờ loại bỏ** khả năng đạt lời giải tối ưu; chín mục của chương làm điều đó bằng ba kiểu lập luận.

Mục	Bài toán	Quy tắc tham lam	Kiểu chứng minh	Thời gian
4.1	Lập lịch khoảng	Kết thúc sớm nhất	Luôn đi trước	O(n log n)
4.1	Phân hoạch khoảng	Gán phòng trống theo thứ tự bắt đầu	Cận dưới cấu trúc (độ sâu)	O(n log n)
4.2	Tối thiểu độ trễ	Hạn chót sớm nhất	Trao đổi (nghịch thế)	O(n log n)
4.3	Bộ nhớ đệm	Dùng lại xa nhất	Trao đổi (lịch rút gọn)	—
4.4	Đường đi ngắn nhất	Đỉnh gần nguồn nhất	Luôn đi trước	O(m log n)
4.5	Cây khung nhỏ nhất	Cạnh rẻ nhất an toàn	Trao đổi (lát cắt, chu trình)	O(m log n)
4.6	Union-Find	Gộp nhỏ vào lớn	Đếm số lần gấp đôi	O(log n) mỗi Find
4.7	Phân cụm	Kruskal dừng ở k cụm	Cận dưới cấu trúc	O(m log n)
4.8	Mã Huffman	Gộp hai ký tự hiếm nhất	Trao đổi + quy nạp	O(k log k)

Mục	Bài toán	Quy tắc tham lam	Kiểu chứng minh	Thời gian
4.9	Cây khung có hướng	Cạnh vào rẻ nhất, co chu trình	Biến đổi chi phí + quy nạp	O(mn)

Bốn bước thiết kế và kiểm chứng một thuật toán tham lam

1. **Đề xuất vài quy tắc tự nhiên** và thử phá chúng bằng phản ví dụ nhỏ (2–3 phần tử). Quy tắc nào sống sót mới đáng chứng minh.
2. **Chọn kiểu chứng minh:** nếu so được hai lời giải "theo từng bước" thì dùng *luôn đi trước*; nếu biến được lời giải tối ưu thành lời giải tham lam bằng các bước đổi nhỏ thì dùng *trao đổi*; nếu tìm được một "vật cản" mà mọi lời giải phải vượt thì dùng *cận dưới cấu trúc*.
3. **Viết chứng minh** chặt chẽ, chú ý điều kiện dùng tới (chi phí không âm ở 4.4, chi phí khác nhau ở 4.5, biết trước tương lai ở 4.3).
4. **Chọn cấu trúc dữ liệu** để mỗi bước tham lam nhanh: sắp xếp, hàng đợi ưu tiên, Union-Find.

Câu hỏi tự kiểm tra

1. Vì sao quy tắc "khoảng ngắn nhất" sai cho lập lịch khoảng? Hãy vẽ phản ví dụ ba khoảng.
   a. *Gợi ý:* [0, 5), [4, 6), [5, 10): khoảng ngắn [4, 6) chặn cả hai khoảng dài.
2. Trong 4.2, vì sao đổi chỗ một nghịch thế **kề nhau** không làm tăng độ trễ lớn nhất?
   a. *Gợi ý:* độ trễ mới của i là f − d_i, nhỏ hơn độ trễ cũ của j là f − d_j vì d_j < d_i.
3. Thêm cạnh s→c độ dài 1 vào đồ thị ở 4.4 thì khoảng cách tới t thay đổi thế nào?
   a. *Gợi ý:* c được chốt ở khoảng cách 1, rồi t ở 1 + 3 = 4.
4. Dijkstra sai ở đâu nếu có cạnh âm? Cho phản ví dụ ba đỉnh.
   a. *Gợi ý:* s→a 2, s→b 3, b→a −2: Dijkstra chốt a = 2 trước, nhưng đường s→b→a chỉ dài 1.
5. Với đồ thị ở 4.5, phân cụm k = 2 có độ giãn cách bao nhiêu?
   a. *Gợi ý:* Kruskal dừng sau khi thêm D–E (còn 2 cụm {A, B, C, D, E}, {F}); cạnh tiếp theo là E–F, độ giãn cách 7.
6. Trong ví dụ Huffman, nếu tần suất e tăng lên 0,20 (và a giảm còn 0,17), cây thay đổi thế nào?
   a. *Gợi ý:* gộp a .17 + d .18 = .35, rồi c .20 + e .20 = .40, rồi b .25 + .35 = .60, cuối cùng .40 + .60. Khi đó b, c, e dài 2 bit, a và d dài 3 bit; ABL = 2 + 0,17 + 0,18 = 2,35 bit.